CS527 – Computer Systems
Lab 8: OpenMP — Arrays and Matrices

Task 1: Array Search: Search for a given element in an array using OpenMP. Determine whether the element exists and, if possible, find its index.

Task 2: Array Sorting: Implement parallel bubble sort or parallelize a suitable sorting algorithm using OpenMP. Explain any synchronization required.

Task 3: Histogram Calculation: Given an array of integers from 0 to 9, count the frequency of each number using OpenMP while preventing race conditions.

Task 4: Matrix Transpose: Write an OpenMP program to compute the transpose of a matrix.

Task 5: 2D convolution: Apply a 3×3 filter to an image represented as a matrix.
2D convolution is an operation in which a small matrix called a filter or kernel slides over a larger matrix, such as an image, and calculates a new value for each position.
It is commonly used in image processing for:

- Blur and smoothing
- Edge detection
- Sharpening images
- Noise reduction

For this problem, we will apply a 3×3 filter to an image represented as a matrix and parallelize the computation using OpenMP.

Example: Suppose we have a 5×5 grayscale image represented by the following matrix:

Input image (5×5)

10	20	30	40	50
60	70	80	90	100
110	120	130	140	150
160	170	180	190	200
210	220	230	240	250

Now, consider this 3×3 filter: Filter (3×3 averaging kernel)

1	1	1
1	1	1
1	1	1

Divide the sum of the 9 values by 9.
Mathematically, the filter is

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

This is an averaging filter that smooths the image.

3. How does convolution work?
The filter is placed over a 3×3 section of the image. Multiply each image value by the corresponding filter value, add the products, and divide by 9.

For example, the first 3×3 region is $\begin{bmatrix} 10 & 20 & 30 \\ 60 & 70 & 80 \\ 110 & 120 & 130 \end{bmatrix}$
The output at the center of this region is

$$O[1][1] = (1*10+1*20+1*30+1*60+1*70+1*80+1*110+1*120+1*130)/9$$
$$= 630/9 = 70$$

The filter then moves one position to the right and repeats the calculation.

For an image of size 5×5, using only valid filter positions (no padding), the output size is
$(5 - 3 + 1) \times (5 - 3 + 1) = 3 \times 3$

What to submit: Checklist
[ ] Source files for all five tasks, each compiling cleanly with `-fopenmp -Wall`
[ ] Correctness verification code/output included for every parallel version
[ ] Timing tables (thread count vs. execution time) for the specified sizes
[ ] Speedup and/or parallel-efficiency numbers where requested